Full Stack Student Management System

Internship Task Report

Student Name: Ghulam Mohiyu Ud Din

Internship Domain: Web Development

Task Number: Task 3

Organization: Devixo Solutions

Email: ghulammohiyudin11@gmail.com

GitHub: github.com/GMD369

LinkedIn: linkedin.com/in/ghulam-mohiyu-ud-din

Date: July 2026

1. Objective

The objective of this task was to design and develop a complete, industry-oriented Student Management System with authentication and database integration, using the MERN stack (MongoDB, Express, React, and Node.js). The application was required to implement user registration, login, and logout backed by secure sessions; a student module supporting add, edit, delete, search, and detailed view operations; a professional dashboard summarising total students and recent registrations; and a responsive, validated user interface with loading indicators. As bonus requirements, the system also needed to store all student records permanently in a database, support pagination on large record sets, and implement role-based login distinguishing Admin and User accounts.

2. Problem Statement

Educational institutes and training organizations commonly rely on scattered spreadsheets or paper records to track student information, which makes searching, updating, and reporting on student data slow and error-prone, and offers no way to control who is allowed to modify records. There was a need for a centralized, database-backed web application where authorised staff (Admins) can safely create, update, and remove student records while other authenticated users can view and search the same data without being able to alter it, all through a clean, responsive interface that works as well on a phone as it does on a desktop, and that scales to hundreds of records without becoming difficult to browse.

3. Technologies Used

- React 19 (Vite) for the single-page frontend
- React Router for client-side routing and route protection
- Axios for API communication with cookie-based credentials
- React Context API for global authentication state
- Node.js and Express for the REST API backend
- MongoDB with Mongoose for schema modelling and persistence (hosted on MongoDB Atlas)
- JSON Web Tokens (JWT) stored in an httpOnly cookie for stateless authentication
- bcryptjs for password hashing
- express-validator for server-side request validation

- Plain CSS3 with custom properties for a themeable, responsive design system (light/dark mode)
- Git and GitHub for version control; Render (API) and Vercel (frontend) for deployment

4. Implementation Steps

1. **Planning the architecture** – decided on a decoupled frontend/backend structure, with React served separately from the Express API in development and communicating over REST, and settled on JWT-in-httpOnly-cookie authentication with two roles, Admin and User.
2. **Building the data models** – designed a `User` schema (name, email, hashed password, role) and a `Student` schema (name, email, roll number, course, phone, address, enrollment date), both with Mongoose, and added a pre-save hook to hash passwords with bcrypt before storage.
3. **Building the authentication API** – implemented register, login, logout, and a session-restore `/me` endpoint, with express-validator rules on every input and a centralized error-handling middleware returning consistent JSON error responses.
4. **Building the student API** – implemented paginated and searchable listing, single-record retrieval, create, update, delete, and a dashboard statistics endpoint, all protected by a `verifyToken` middleware, with create/update/delete additionally gated by a `requireRole('admin')` middleware.
5. **Building the React shell** – created an `AuthContext` that restores the session on page load, a `ProtectedRoute` component that redirects unauthenticated users to the login page, and a shared dashboard layout with a sidebar and topbar.
6. **Building the UI features** – implemented the Login and Register pages with client-side validation, the Dashboard with stat cards and a recent-registrations panel, and the Students page with a debounced search box, a paginated data table, and modal forms for adding, editing, and viewing student details, with admin-only controls hidden for non-admin users.
7. **Designing the visual theme** – built a token-based design system (colors, spacing, shadows, radii) supporting both light and dark mode automatically, an icon-based sidebar, avatar and role badges, and a fully responsive layout that collapses the sidebar into a slide-out drawer and the data table into stacked cards on mobile.
8. **Seeding and testing data** – wrote a seed script to populate the database with 200 realistic student records so pagination, search, and dashboard statistics could be verified against a non-trivial dataset.

9. **Deployment** – deployed the Express API to Render and the React build to Vercel, and resolved the resulting cross-origin cookie and CORS configuration required for the two to communicate securely in production.

5. Code Screenshots with Explanations

5.1 JWT Auth Middleware (server/middleware/auth.js)

```
async function verifyToken(req, res, next) {
  const token = req.cookies?.token;
  if (!token) return res.status(401).json({ message: 'Not authenticated' });

  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  const user = await User.findById(decoded.id).select('-password');
  if (!user) return res.status(401).json({ message: 'User no longer exists' });

  req.user = user;
  next();
}

function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Insufficient permissions' });
    }
    next();
  };
}
```

This middleware reads the JWT from the httpOnly cookie set at login, verifies it, and attaches the authenticated user to the request. `requireRole` is layered on top of it on the student create, update, and delete routes so that only Admin accounts can modify data, while both roles can still read it — enforcing the same rule the client UI already hides behind role checks, but on the server where it cannot be bypassed.

5.2 Paginated & Searchable Student Listing (server/controllers/studentController.js)

```
async function listStudents(req, res) {
  const page = Math.max(parseInt(req.query.page, 10) || 1, 1);
  const limit = Math.min(Math.max(parseInt(req.query.limit, 10) || 10, 1), 100);
  const q = (req.query.q || '').trim();

  const filter = q ? { $or: [
    { name: { $regex: q, $options: 'i' } },
    { email: { $regex: q, $options: 'i' } },
    { rollNumber: { $regex: q, $options: 'i' } },
    { course: { $regex: q, $options: 'i' } },
  ] } : {};
```

```
  const [data, total] = await Promise.all([
    Student.find(filter).sort({ createdAt: -1 })
      .skip((page - 1) * limit).limit(limit),
    Student.countDocuments(filter),
  ]);

  res.json({ data, total, page, pages: Math.ceil(total / limit) || 1 });
}
```

The listing endpoint combines pagination and search in a single query: an optional case-insensitive regex filter matches against name, email, roll number, or course, and `skip`/`limit` compute the correct page slice. The response returns the total record count and total page count alongside the page of data, which the React Students page uses to render its pagination controls.

5.3 Auth Session Restore (client/src/context/AuthContext.jsx)

```
useEffect(() => {
  api.get('/auth/me')
    .then((res) => setUser(res.data.user))
    .catch(() => setUser(null))
    .finally(() => setLoading(false));
}, []);
```

On first load, the React app calls the `/auth/me` endpoint with credentials attached; the httpOnly cookie set during login is sent automatically, so a page refresh keeps the user signed in without storing any token in client-side JavaScript, avoiding exposure to XSS-based token theft. `ProtectedRoute` reads this context and redirects to `/login` whenever `user` is null once loading completes.

5.4 Role-Gated UI Controls (client/src/pages/Students.jsx)

```
{isAdmin && (
  <>
    <button onClick={() => setEditing(s)}><EditIcon /></button>
    <button onClick={() => handleDelete(s)}><TrashIcon /></button>
  </>
)}
```

Edit and delete controls in the student table are only rendered when the signed-in user's role is `admin`. This is a UX convenience, not a security boundary — the same rule is independently enforced server-side by `requireRole('admin')`, so a User account cannot perform a mutation even by calling the API directly.

6. Output / Result Screenshots

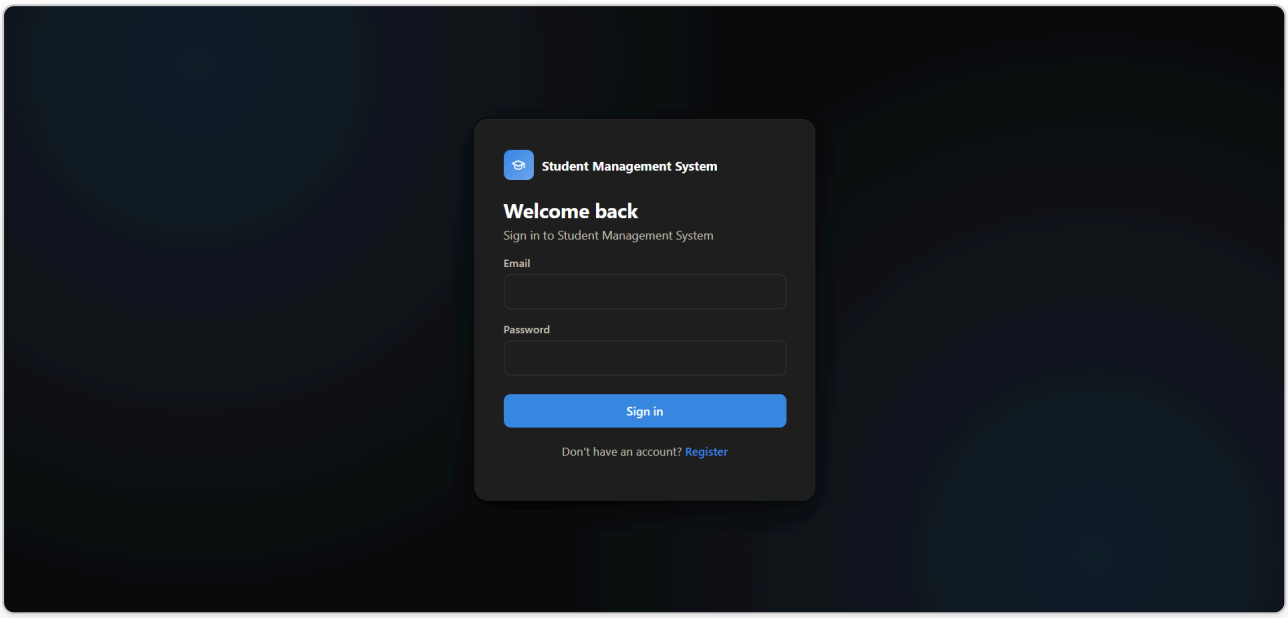

Figure 6.1 – Login page, showing the branded auth card with email and password fields, client-side validation, and a link to the registration page.

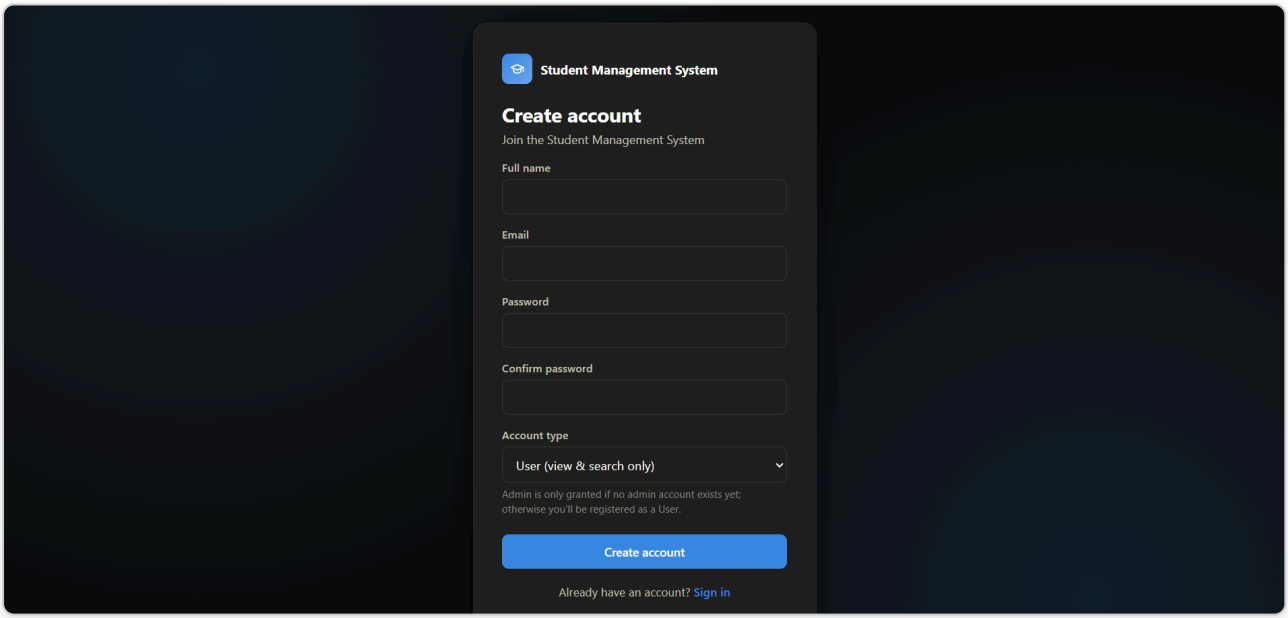

Figure 6.2 – Registration page, showing the account creation form with full name, email, password, confirm password, and an account-type selector for Admin/User role.

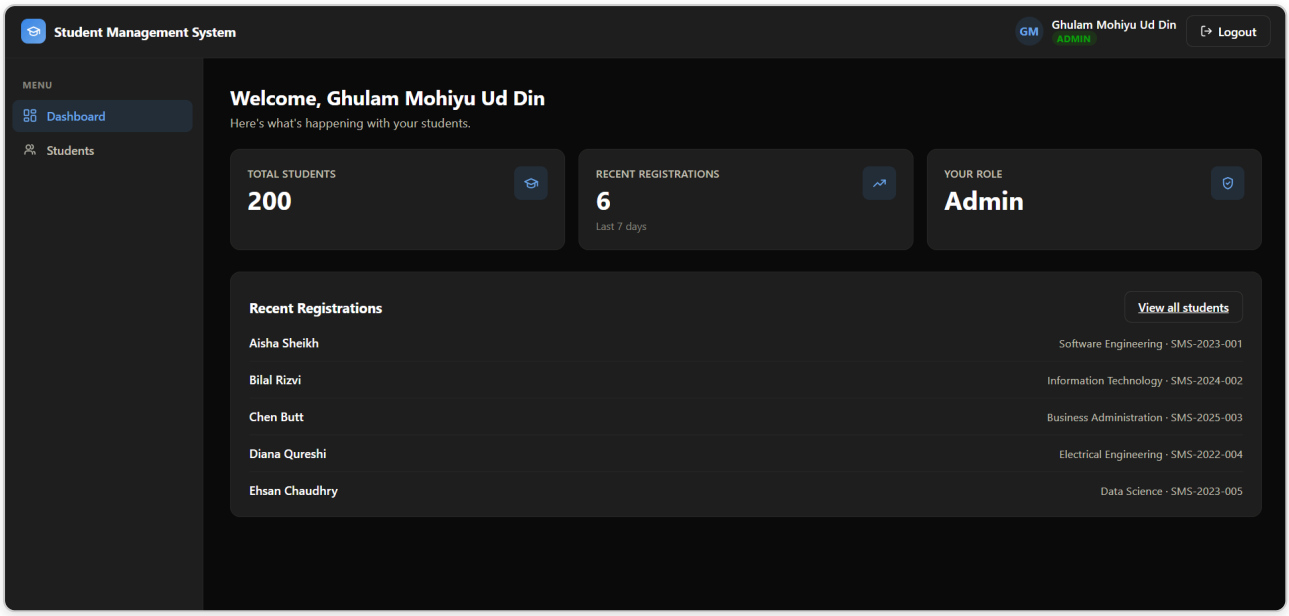

Figure 6.3 – Dashboard, showing stat cards for Total Students, Recent Registrations, and the signed-in user's role, alongside a Recent Registrations panel.

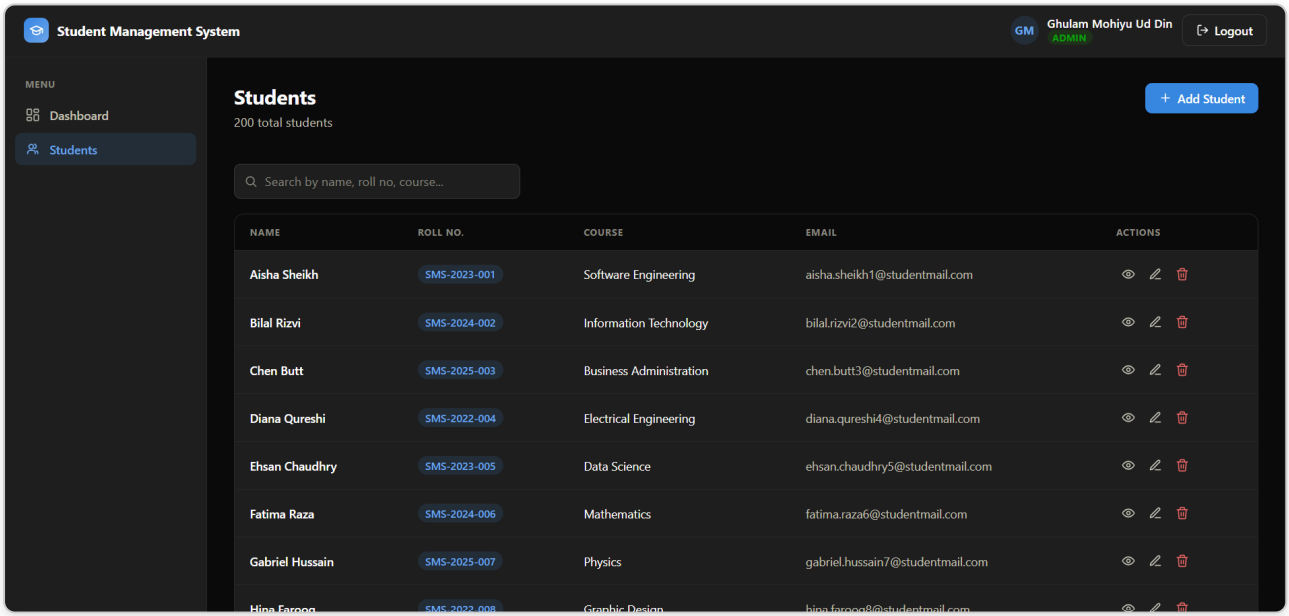

Figure 6.4 – Students page, showing the paginated, searchable data table of 200 seeded records with roll-number chips and admin-only row actions (view, edit, delete).

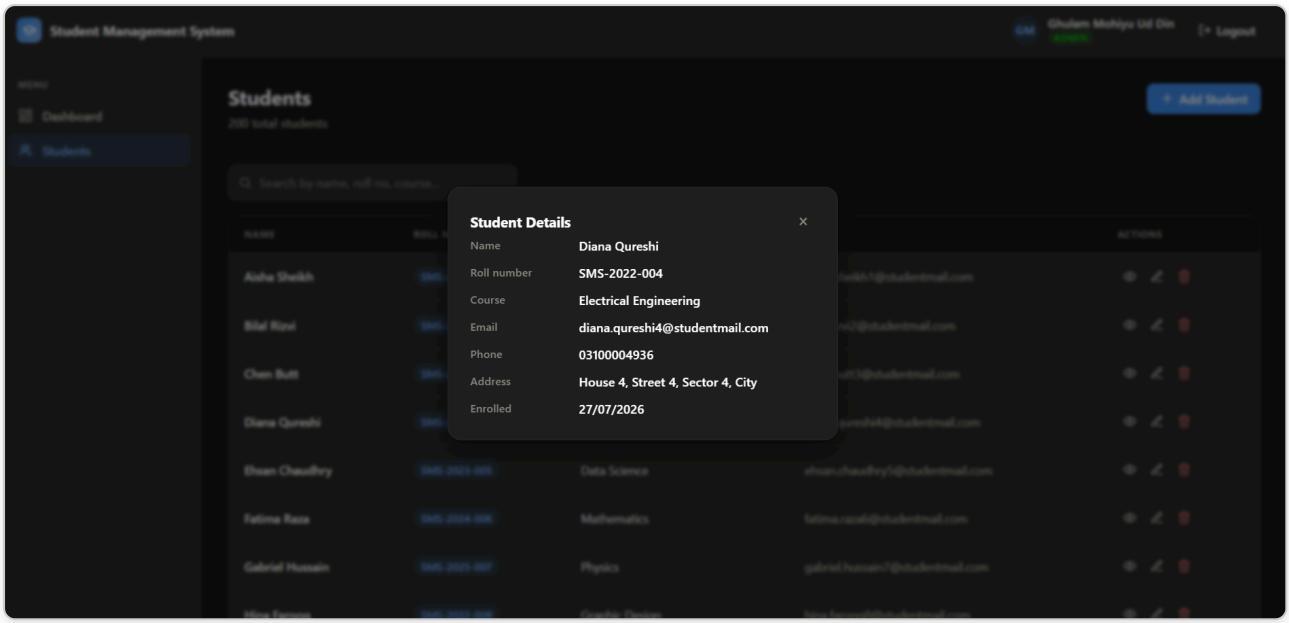

Figure 6.5 – Student Details modal, showing the full record – name, roll number, course, email, phone, address, and enrollment date – opened from the Students table.

7. Challenges Faced

- **Cross-origin authentication cookies** – once the frontend (Vercel) and backend (Render) were deployed to different domains, the login cookie stopped being sent with API requests; this was traced to the default `SameSite=Lax` cookie attribute, which browsers do not attach to cross-site XHR/fetch requests, and was fixed by switching to `SameSite=None; Secure` in production.
- **CORS configuration in production** – the deployed API rejected requests from the Vercel origin with no `Access-Control-Allow-Origin` header; this required moving from a single hard-coded allowed origin to a comma-separated allow-list read from an environment variable, and confirming the exact deployed frontend origin was added to it on Render.
- **Enforcing role-based access on both layers** – hiding Admin-only buttons in the React UI was not sufficient by itself, since a non-admin user could still call the API directly; every mutating student route also needed its own server-side role check.
- **Debounced search without extra requests** – typing in the search box naively would fire an API request per keystroke; this was solved with a 350ms debounce in the SearchBar component before the query state (and therefore the API call) updates.
- **Seeding realistic test data** – verifying pagination and dashboard statistics required more than a handful of records, so a standalone seed script was written to generate 200 students with varied enrollment dates, some falling within the last 7 days to populate the "Recent Registrations" stat correctly.

- **Consistent theming across light and dark mode** – every surface, border, and text color needed to route through CSS custom properties rather than hard-coded hex values, so the entire dashboard, forms, and tables could re-theme correctly under the system's dark mode.

8. Conclusion

This task was completed successfully, delivering a full-stack Student Management System built with React, Node.js, Express, and MongoDB. All required features were implemented, including authentication (register, login, logout), a complete student module (add, edit, delete, search, view details), a professional dashboard with live statistics, and a responsive interface with client- and server-side form validation and loading indicators. Both bonus requirements were also delivered: student records are stored permanently in MongoDB, and the system supports pagination alongside role-based login distinguishing Admin and User accounts, with access control enforced independently on the client and the server. The application was deployed end-to-end, with the API hosted on Render and the frontend on Vercel, and the cross-origin authentication issues that surfaced during deployment were diagnosed and resolved. This project reinforced practical, production-relevant skills in secure cookie-based authentication, role-based authorization, REST API design with pagination and search, and responsive full-stack application deployment.